Technische Universität Braunschweig

Chair for Chip Design for Embedded Computing

Hardware-Software-Systeme

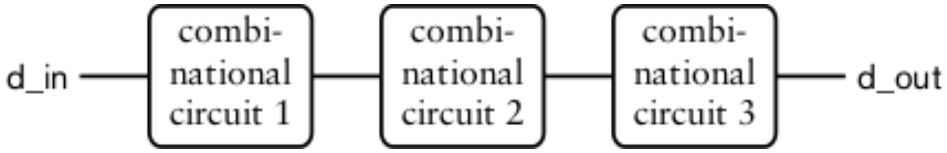
Kapitel 03 - VHDL - Hardware-Beschreibungssprache

Prof. Dr.-Ing. Guillermo Payá Vayá, 19. November 2024

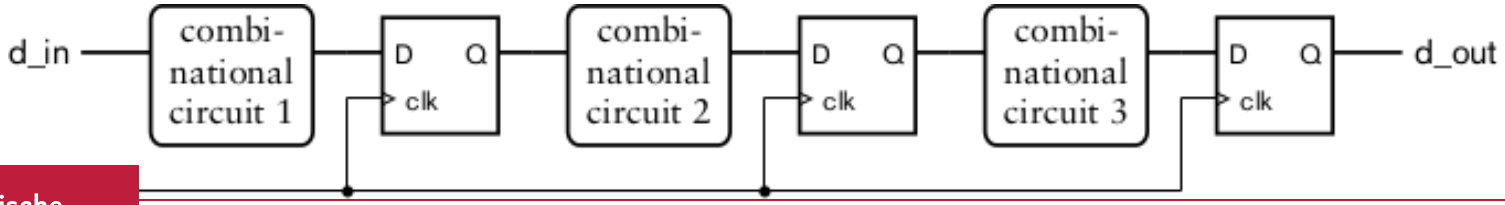
Sequenzielle Schaltungen

Kombinatorische und Sequenzielle Schaltungen

- **Kombinatorische Schaltungen**: Die Ausgangswerte der Schaltung zu einem bestimmten Zeitpunkt werden ausschließlich durch die Kombination der Eingangswerte zu diesem Zeitpunkt bestimmt.
  $\rightarrow$ Die Speicherung von Informationen findet nicht statt, d. h. es gibt keine Abhängigkeit von Werten zu früheren Zeitpunkten.



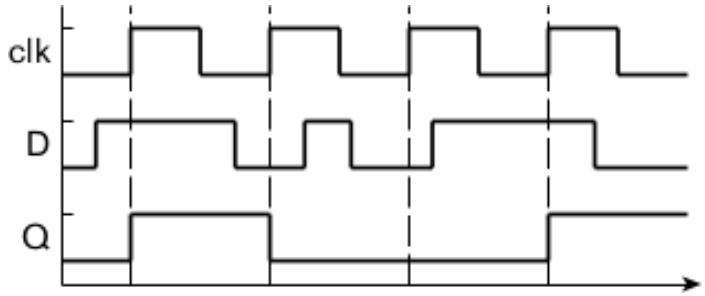
- **Sequentielle Schaltungen**: Sie enthalten Speicher, die es ermöglichen, die Ausgangswerte sowohl durch die aktuellen als auch durch frühere Eingangswerte zu bestimmen.



Taktflankengesteuertes D-Flipflops (I)

- Bei der VHDL-Beschreibung taktflankengesteuerter Elemente ist auf die Syntax und auf die Sensitivity-List besonders zu achten
- Beispiel: D-FF mit asynchronem Reset

```
dff1: process (clk, reset_n)
begin
  if reset_n = '0' then
    Q <= '0';
  elsif clk = '1' and clk'event then
    Q <= D;
  end if;
end process dff1;
```



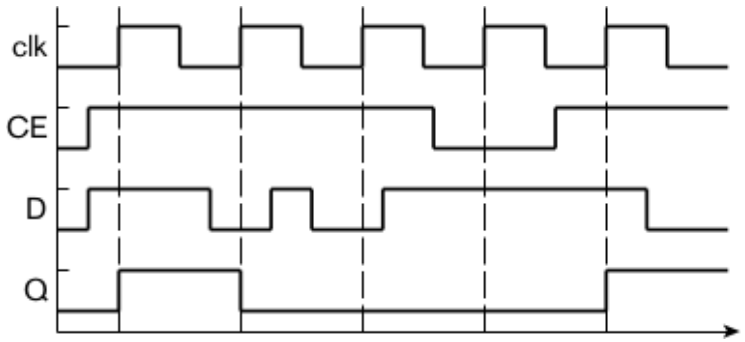
D-FF 1

- Der Eingang ist nicht in der Sensitivity-List, denn, nur wenn sich der reset oder der Takt ändern, soll der Prozess ausgeführt werden
- Der Ausdruck (`clk='1' and clk'event` bezeichnet die Taktflanke (`'1'` bei der steigenden, `'0'` bei der fallenden)
  - alternativer Ausdruck: `rising_edge(clk)` bzw. `falling_edge(clk)`.

Taktflankengesteuertes D-Flipflops (II)

- Beispiel: D-FF mit synchronem `reset` und `clock enable (CE)`

```
dff2: process (clk)
begin
  if clk = '1' and clk'event then
    if reset_n = '0' then
      Q <= '0';
    elsif CE = '1' then
      Q <= D;
    end if;
  end if;
end process dff2;
```



D-FF 2

- Diesmal reagiert der Prozess ausschließlich auf einer Änderung im Taktsignal
- Nur wenn im Moment der steigenden Taktflanke das `clock-enable`-Signal aktiv ist, erhält `Q` den Wert von `D`, sonst behält `Q` seinen Wert, ungeachtet dessen, was mit `D` passiert

Pitfalls: Halten von Signalen

- Wenn bestimmte Zustände zu einer Änderung des zu speichernden Werts führen sollen und sonst der Wert gespeichert werden soll:
  - außerhalb des sequenziellen Prozesses dem zukünftigen Signalwert den aktuellen Signalwert standardmäßig zuweisen
  - oder mit einem erzeugten `enable`-Signal eine Übernahme des neuen Werts steuern

```
process (sList)
begin
  if cond1 then
    f <= expr1;
  elsif cond2 then
    f <= expr2;
  end if;
end process;
.
```

```
process (f, sList)
begin
  f_next <= f;
  if cond1 then
    f_next <= expr1;
  elsif cond2 then
    f_next <= expr2;
  end if;
end process;
process (clk)
begin
  if rising_edge(clk) then
    f <= f_next;
  end if;
end process;
```

Pitfall- Halten von Signalen

Variablen in taktsynchronen Prozessen (I)

- Zum Verständnis der Funktion von Variablen in taktsynchronen, synthetisierbaren Prozessen ist insbesondere zu analysieren, ob diese zu kombinatorischer Logik oder zu FFs synthetisiert werden.

(1) Variablen, die in einer taktgesteuerten Umgebung in jedem Fall eine Wertzuweisung erfahren, bevor sie gelesen werden, werden zu kombinatorischer Logik synthetisiert oder rausoptimiert

```
entity VAR_TEST is
  port (A, B, C, clk : in std_ulogic;
        Q1           : out std_ulogic);
end VAR_TEST;
architecture RTL1 of VAR_TEST is
begin
  P1: process (clk)
    variable TEMP: std_ulogic;
  begin
    if clk='1' and clk'event then
      TEMP := A and B;
      Q1 <= TEMP and C;
    end if;
  end process P1;
end architecture RTL1;
```

Variablen 1

Variablen in taktsynchronen Prozessen (II)

- Zum Verständnis der Funktion von Variablen in taktsynchronen, synthetisierbaren Prozessen ist insbesondere zu analysieren, ob diese zu kombinatorischer Logik oder zu FF synthetisiert werden.

(2) Wenn sie hingegen zuerst gelesen werden, bevor ihnen ein Wert zugewiesen wird, so werden daraus Flipflops generiert.

```
entity VAR_TEST is
  port (A, B, C, clk : in std_ulogic;
        Q1           : out std_ulogic);
end VAR_TEST;
architecture RTL2 of VAR_TEST is
begin
  P1: process (clk)
    variable TEMP: std_ulogic;
  begin
    if clk='1' and clk'event then
      Q1 <= TEMP and C;
      TEMP := A and B;
    end if;
  end process P1;
end architecture RTL2;
```

Variablen 2

Taktflankengesteuertes Register

- Taktflankengesteuerte Register lassen sich nach denselben Grundregeln entwerfen wie Flipflops.
- Im Unterschied zum D-Flipflop werden jedoch als Datenein- und ausgänge Signale vom Datentyp `std_ulogic_vector` verwendet.

```
entity REG_GEN is
  generic (N : integer := 16);
  port (clk, reset_n : in  std_ulogic;
        D            : in  std_ulogic_vector(N-1 downto 0);
        Q            : out std_ulogic_vector(N-1 downto 0));
end REG_GEN;
architecture RTL of REG_GEN is
begin
  REG: process (clk, reset_n)
  begin
    if reset_n = '0' then
      Q <= (others => '0');
    elsif clk = '1' and clk'event then
      Q <= D;
    end if;
  end process REG;
end architecture RTL;
```

Register

Parametrisiertes N-Bit Schieberegister

```
entity SRG_PAR is
  generic (WIDTH : natural := 4;  -- Registerbreiter
           SHW   : natural := 2); -- Schiebeweite
  port (clk, LOAD, R_L : in  std_ulogic;
        D_in           : out std_ulogic_vector(WIDTH-1 downto 0);
        Y_out          : out std_ulogic_vector(WIDTH-1 downto 0));
end SRG_PAR;
architecture RTL of SRG_PAR is
  signal Y_int : std_ulogic_vector(WIDTH-1 downto 0);
  signal TEMP  : std_ulogic_vector(SHW-1 downto 0);
begin
  TEMP <= (others => '0');
  SYN_SHIFT : process (clk)
  begin
    if clk = '1' and clk'event then
      if LOAD = '1' then
        Y_int <= D_in;
      elsif R_L = '0' then -- links schieben
        Y_int <= Y_int(WIDTH-1-SHW downto 0) & TEMP;
      else                 -- rechts schieben
        Y_int <= TEMP & Y_int(WIDTH-1 downto SHW);
      end if;
    end if;
  end process SYN_SHIFT;
  Y_out <= Y_int;
end architecture RTL;
```

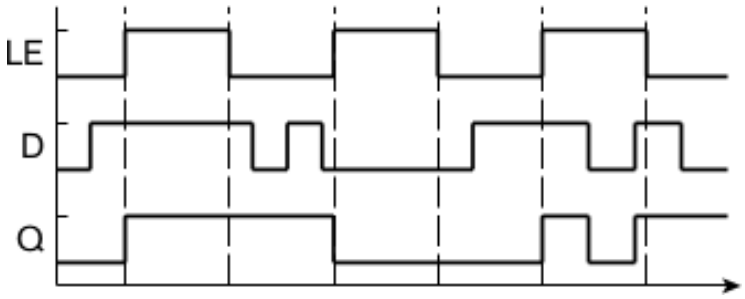
Schiebe-register

Erzeugung von Latches (I)

- Latches, also taktzustandsgesteuerte Flipflops, dienten in digitalen Schaltungen zur Speicherung von Signalzuständen:
  - → Während des aktiven Taktpegels, also bei geöffneten Taktor, ist das Latch transparent, der Wert des Dateneingangs erscheint am Ausgang.
  - → Während des passiven Taktpegels wird der zuletzt am Dateneingang liegende Signalwert gespeichert.
- Das Speicherverhalten von Signalen und Variablen in Prozessen, die keine Taktflankenabfrage besitzen, wird in VHDL auf zweierlei Weise realisiert:
  - Wenn in einer `if-then-else-/case-`Anweisung einem Signal oder einer Variablen in mindestens einer Verzweigung kein Wert zugewiesen wird (unvollständige Signalverzweigung)
  - Wenn ein Signal oder eine Variable auf mindestens einem Weg durch einen Prozess bzw. eine nebenläufige Anweisung gelesen wird, bevor eine Wertzuweisung erfolgt

Erzeugung von Latches (II)

```
entity LATCH is
  port (D, clk : in std_ulogic;
        Q1, Q2 : out std_ulogic);
end LATCH;
architecture RTL of LATCH is
begin
  L1: process (clk, D)
  begin
    if clk='1' then
      Q1 <= D;
    end if;
  end process L1;
  L2: process (clk, D)
  begin
    case clk is
      when '1' => Q2 <= D,
      when others => null;
    end case;
  end process L2;
end architecture RTL;
```



Latch 1

Erzeugung von Latches (III)

```
entity LATCH2 is
  port (D, clk : in std_ulogic;
        Q1, Q2 : out std_ulogic);
end LATCH2;
architecture RTL of LATCH2 is
  signal TEMP : std_ulogic;
begin
  L3:process(clk,D)
  begin
    if clk='1' then
      Q1 <= TEMP;
    else
      TEMP <= D;
    end if;
  end process L3;
  L4:process(clk,D)
    variable TEMP : std_ulogic;
  begin
    if clk='1' then
      TEMP := D;
    end if;
    Q2 <= TEMP;
  end process L4;
end architecture RTL;
```

Latch 2

RTL-Syntheserichtlinien

- In einem Schaltwerk sollten die Speicherelemente und das Schaltnetz in getrennten Prozesse geschrieben werden
  - Bessere Lesbarkeit, einfachere Fehleranalyse
- Im Bezug auf kombinatorische Prozesse
  - Alle Signale, die einen Wert zugewiesen bekommen (Ausgangssignale), müssen unter jeder möglichen Eingangsbedingung einen Wert erhalten
    $\rightarrow$ sonst werden ungewollte pegelgesteuerte Speicherelemente erzeugt
  - Kein Ausgangssignal sollte im Prozess gelesen werden
    $\rightarrow$ Bei Unvorsichtigkeit besteht die Gefahr einer komb. Rückkopplung
- Im Bezug auf sequenzielle Prozesse
  - Sequenzielle Prozesse dürfen nur von einem **einzigen** Takt und evtl. auch von asynchronen Reset/Set-Signalen abhängig sein
  - Lokale Taktsignale sollen **nie** erzeugt bzw. verwendet werden
    $\rightarrow$ `Enables` erfüllen mit Sicherheit ebenfalls den gewünschten Zweck

Struktureller VHDL Entwurf (Prozesse, Funktionen und Packages)

Strukturelle Beschreibung

- Die strukturelle VHDL-Beschreibung eines Systems definiert, wie ein System aufgebaut ist und wie die Komponenten und/oder Prozesse miteinander und mit der Umwelt verbunden sind
- Strukturelle Beschreibung erlaubt mehrere Hierarchieebenen
  $\rightarrow$ Eine `Entity` kann aus mehreren Komponenten bestehen, die wiederum strukturell oder verhaltensmäßig beschrieben worden sind
- In einer *rein strukturellen Beschreibung* gibt es keine Verhaltensbeschreibungen
  - keine Prozesse
  - keine komplexen, nebenläufigen Ausdrücke
  - Die tiefste Hierarchieebene besteht aus technologischen Primitiven
- Der ideale VHDL-Entwurf ist eine Mischung aus struktureller (Top- und Subsystem-Ebene) und funktionaler Beschreibung

Komponentendeklaration

- Um Einheiten in einer Architektur einbinden zu können, muss es einen Platzhalter für sie geben: Ein `component` wird hierfür verwendet. Seine Deklaration befindet sich
  - entweder im deklarativen Abschnitt der instanziierenden Architektur
  - oder in einem eingebundenem `Package`
- Syntax:

```
component component_name
  generic ( generic_list );
  port    ( port_list );
end component component_name; -- Name hier optional
```

- Obwohl nicht zwingend erforderlich, sollte ein `component` denselben Namen tragen wie die entsprechende `entity` und über dieselben Schnittstellen (im Name und Datentyp) verfügen
  $\rightarrow$ Nur so ist eine automatische Zuordnung seitens des Tools möglich

Komponenteninstanziierung (I)

- In der `architecture` werden die deklarierten Komponenten instanziiert und über Signale mit den restlichen Bestandteilen der Architektur verbunden
  - Bei passender Richtung (und Typ und Breite) ist eine direkte Verbindung mit den Ports der instanziierenden `entity` erlaubt
- Ein `component` darf mehrmals instanziiert werden
- Syntax:

```
instance_label : component_name
  generic map ( generic_map_anweisung )  -- kein ';'
  port map ( port_map_anweisung );
```

- Eine `map_anweisung` ist eine mit Kommata getrennte Liste, wo den `component`-Schnittstellen bestimmte in der instanziierenden `architecure` sichtbare Objekte zugewiesen werden
- Werden den `generics` keine Werte zugewiesen, gelten die bei der Entity-Deklaration angegebenen Werte

Komponenteninstanziierung: Beispiel

```
library IEEE;
use IEEE.std_logic_1164.all;
entity ADDER is
  port ( X, Y   : in  std_ulogic_vector(7 downto 0);
         Res    : out std_ulogic_vector(7 downto 0));
end ADDER;
architecture struct of ADDER is
  component FA is
    port ( Op1, Op2, Ci : in  std_ulogic;
           Res, Co      : out std_ulogic);
  end component;
  constant LOW : std_ulogic := '0';
  signal carry : std_ulogic_vector(X'range);
begin
  FA_0 : FA port map(X(0), Y(0), LOW,
                     Res(0), carry(1));
  FA_1 : FA port map(Res => Res(1),
                     Ci => carry(1),
                     Op1 => X(1),
                     Op2 => Y(1),
                     Co => carry(2));
  ...
end architecture struct;
```

Komponenten-intanziierung

Komponenteninstanziierung (II)

- Nicht benutzte **Komponentenausgänge** können offen gelassen werden, indem der entsprechende Klemme als Actual Port das Schlüsselwort `open` zugeordnet wird.
- Unbenutzte **Eingänge** einer Komponente hingegen sind vollständig zu beschalten, damit keine undefinierten Pegel entstehen. Hierzu muss ggf. eine zusätzlich zu deklarierendes, internes Signal an die jeweilige Komponentenklemme gelegt werden.

```
ZERO <= '0';
FA_0 : FU port map(Res => Res(0),
                   Ci => ZERO,
                   Op1 => X(0),
                   Op2 => Y(0),
                   Co => open);
```

Iterative Instanziierung: `generate`-Ausdrücke

- Die sog. `generate`-Ausdrücke erhöhen die Lesbarkeit und die Wiederverwendbarkeit einer VHDL-Modulbeschreibung
  - Bei hoher HW-Redundanz, z.B. viele Instanzen eines Moduls, kann ein `for-generate` Konstrukt die Lesbarkeit erhöhen
  - Bei hochparametrisierten Beschreibungen, z.B. wenn die Top-Ebene einer Systembeschreibung mehrere Implementierungsalternativen beinhalten soll, ist ein `if-generate` Konstrukt unerlässlich

```
identifier_1 : for Parameter in Wert_1 to Wert_2 generate
               identifier_2 : if Bedingung generate
                              Komponenteninstanziierung
                              end generate identifier_2;
               end generate identifier_1;
```

→ Der `Parameter` im `generate`-Schema ist eine statische Größe, die hier implizit deklariert wird, nur innerhalb der `generate`-Anweisung verfügbar ist und nicht verändert werden darf.

→ Die Bedingung in der `if generate`-Anweisung fragt den Wert des Parameters ab und erzeugt ggf. eine spezielle Instanziierung.

Iterative Instanziierung: Beispiel

```
library IEEE;
use IEEE.std_logic_1164.all;
entity ADDER is
  generic (BITS : integer);
  port ( X, Y   : in  std_ulogic_vector(BITS-1 downto 0);
         Res    : out std_ulogic_vector(BITS-1 downto 0));
end ADDER;
architecture struct of ADDER is
  component FA is
    port ( Op1, Op2, Ci : in  std_ulogic;
           Res, Co      : out std_ulogic);
  end component;
  constant LOW : std_ulogic := '0';
  signal carry : std_ulogic_vector(X'range);
begin
  RCA  : for i in 0 to BITS-1 generate
         GEN_FA_0 : if (i = 0) generate
                    FA_0 : FA port map(X(i), Y(i), LOW,
                                       Res(i), carry(i));
                    end generate GEN_FA_0;
         GEN_FA_i : if (i > 0) generate
                    FA_i : FA port map(X(i), Y(i), carry(i-1),
                                       Res(i), carry(i));
                    end generate GEN_FA_i;
         end generate RCA;
end architecture struct;
```

Iterative intanziierung

Konfiguration zu Auswahl von Architekturen

- Ein sog. `configuration` ist erforderlich
  - Beim Vorhandensein mehrerer Architekturen für eine `entity`, da sonst die Synthese- bzw. Simulationstools sich irgendeine Architektur aussuchen
  - Bei Unterschieden zwischen Komponenten und den entsprechenden `entities`, da sonst die Werkzeuge keine Hierarchie aufbauen können
- Die `configuration` kann nach der `architecture` oder in einer separaten Datei definiert werden
- Werden nur bei komplexen Systemhierarchien mit mehreren Abstraktionsebenen oder Realisierungsalternativen gebraucht
- *Wird nicht in allen Tools gleich unterstützt, der Identifier kann unterschiedlich sein.*

```
for identifier | others | all : component_name
  use entity libray.entity_name(architecture_name);
```

Konfiguration: Beispiel

```
library IEEE;
use IEEE.std_logic_1164.all;
entity ADDER is
  generic (BITS : integer);
  port ( X, Y   : in  std_ulogic_vector(BITS-1 downto 0);
         Res    : out std_ulogic_vector(BITS-1 downto 0));
end ADDER;
architecture struct of ADDER is
  component FA is
    port ( Op1, Op2, Ci : in  std_ulogic;
           Res, Co      : out std_ulogic);
  end component;
  constant LOW : std_ulogic := '0';
  signal carry : std_ulogic_vector(X'range);
  -- Konfigurationen
  for FA_0 : FA use entity work.FA(behave);
  for others : FA use entity work.FA(struct);
begin
  RCA  : for i in 0 to BITS-1 generate
         GEN_FA_0 : if (i = 0) generate
                    FA_0 : FA port map(X(i), Y(i), LOW,
                                       Res(i), carry(i));
                    end generate GEN_FA_0;
         GEN_FA_i : if (i > 0) generate
                    FA_i : FA port map(X(i), Y(i), carry(i-1),
                                       Res(i), carry(i));
                    end generate GEN_FA_i;
  ...
```

Konfiguration

Unterprogramme in VHDL (I)

- Wie bei SW-Programmiersprachen, ist in VHDL die Deklaration und Benutzung von Unterprogrammen (d.h. `Funktionen` und `Prozeduren`) möglich
- Eigenschaften
  - Die Ausführung erfolgt sequenziell
  - Orte der Deklaration
    - In Architekturen → im deklarativen Teil der Architektur werden die Unterprogramme direkt beschrieben
    - In Packages → im deklarativen Teil des Package werden nur die Prototypen der Unterprogramme deklariert (analog zu einem Header-File in C). Im Körper des Package werden die Unterprogramme beschrieben

```
function Name (ArgumentList)
  return rueckgabetyp is
begin
  ... -- sequenzielle anweisungen
  return Rueckgabeausdruck;
end function Name;
```

```
procedure Name (ArgumentList) is
  ... -- variable deklarationen
begin
  ... -- sequenzielle anweisungen

end procedure Name;
```

Unterprogramme in VHDL (II)

- Die Zuordnung der aktuellen Argumente erfolgt entweder durch Angabe in der richtigen Reihenfolge (Position Order) oder aber durch Angabe der formalen Argumentnamen (Named Order). Ähnlich wie bei der Instanziierung von Komponenten.
- Die Aktivierung eines Unterprogramms beginnt mit der Wertänderung eines aktuellen Arguments. Sie ist also vergleichbar zur Aktivierung eines Prozesses mit Sensitivity Liste.
- Innerhalb von Unterprogrammen dürfen Variablen verwendet werden. Allerdings wird deren Wert, im Gegensatz zu Prozessen, beim Verlassen des Unterprogramms **nicht** bis zum nächsten Aufruf gespeichert, sondern bei jedem Aufruf neu initialisiert.
- Signaldeklarationen sind hingegen in Unterprogrammen nicht synthesefähig.
- Unterprogrammen dürfen weitere Unterprogramme aufrufen.

Unterprogramme: `function`

```
entity PARITAET is
  generic (BITS : integer := 4);
  port (D       : in std_ulogic_vector(BITS-1 downto 0);
        GERADE  : out std_ulogic);
  end PARITAET;
architecture RTL of PARITAET is
  function PARITY_FUNC (X : std_ulogic_vector(BITS-1 downto 0))
    return std_ulogic is
    variable GERADE : std_ulogic := '0';
  begin
    for I in BITS-1 downto 0 loop
      if (X(I) = '1') then
        GERADE := not GERADE;
      end if;
    end loop;
    return GERADE;
  end function PARITY_FUNC;
begin
  GERADE <= PARITY_FUNC(D);
end architecture RTL;
```

Function

Unterprogramme: `procedure`

```
entity PARITAET is
  generic (BITS : integer := 4);
  port (D       : in std_ulogic_vector(BITS-1 downto 0);
        GERADE  : out std_ulogic);
  end PARITAET;
architecture RTL of PARITAET is
  procedure PARITY_PROC (
    signal X   : in  std_ulogic_vector(BITS-1 downto 0);
    signal PAR : out std_ulogic) is
    variable par_int : std_ulogic := '0';
  begin
    for I in BITS-1 downto 0 loop
        if (X(I) = '1') then
          par_int := not par_int;
        end if;
    end loop;
    PAR <= par_int;
  end procedure PARITY_PROC;
begin
  PAR_INST: process (D)
  begin
    PARITY_PROC(X   => D,
                PAR => GERADE);
  end process PAR_INST;
end architecture RTL;
```

Procedure

Definition von `packages`

- Verschiedene, in einem Entwurfsprojekt gemeinsam verwendete Deklarationen und Unterprogramme sollten in einem `package` zusammengefasst werden.
- Ein `package` kann die nachfolgenden VHDL-Objekte enthalten:
  - constant-, type- und subtype-Deklarationen
  - Komponentendeklarationen
  - Funktionen und Prozeduren
- Durch Verwendung eines package kann die Übersichtlichkeit eines komplexen VHDL-Entwurfs deutlich verbessert werden:
  - Datentypen und Unterprogrammdeklarationen, die ohne package in verschiedenen Entwurfseinheiten einheitlich deklariert werden müssten, werden in einer einzigen Entwurfseinheit abgelegt.
  - Es kann auf andere, bereits erfolgreich implementierte Entwurfseinheiten, ggf. auch von externen Designhäusern, zurückgegriffen werden (z.B. IEEE-Standard `std_logic_1164`)

Einsatz von `packages`

- Die Verwendung eines `package` erfordert eine `package`-Deklaration sowie ggf. einen `package body`.
- Während in der Deklaration die Namen bzw. Werte aller Konstanten, Datentypen und Unterprogramme mit Ihren Schnittstellen bzw. deren Typen spezifiziert werden, enthält der Körper die funktionalen Inhalte der Unterprogramme.

```
[<library und use Ausdruecke>]
package Package_Name is
<Deklarationsliste>
end Package_Name;

[<library und use Ausdruecke>]
package body Package_Name is
<Definitionsliste>
end Package_Name;
```

Einsatz von packages: Beispiel (I)

```
library IEEE;
use IEEE.std_logic_1164.all;

package paritaet_pkg is
  procedure PARITY_PROC (signal X   : in  std_ulogic_vector;
                         signal PAR : out std_ulogic);
end paritaet_pkg;
package body paritaet_pkg is
  procedure PARITY_PROC (signal X   : in  std_ulogic_vector;
                         signal PAR : out std_ulogic) is
  variable par_int : std_ulogic := '0';
  begin
    for I in X'length-1 downto 0 loop
      if (X(I) = '1') then
        par_int := not par_int;
      end if;
    end loop;
    PAR <= par_int;
  end procedure PARITY_PROC;
end paritaet_pkg;
```

Einsatz von packages: Beispiel (II)

```
library IEEE;
use IEEE.std_logic_1164.all;

library work;
use work.paritaet_pkg.all;

entity PARITAET is
  generic (BITS : integer := 4);
  port (D      : in  std_ulogic_vector(BITS-1 downto 0);
        GERADE : out std_ulogic);
end PARITAET;

architecture RTL of PARITAET is
begin
  PAR_INST: process (D)
  begin
    PARITY_PROC(X => D, PAR => GERADE);
  end process PAR_INST;
end architecture RTL;
```

Package

IP-zentrierte Entwurfsmethode

Hersteller stellen Hardware-Module als IPs (intellectual property) zur Verfügung oder verkaufen sie. Der Source-Code von IPs kann dabei verborgen werden.

IP Catalog

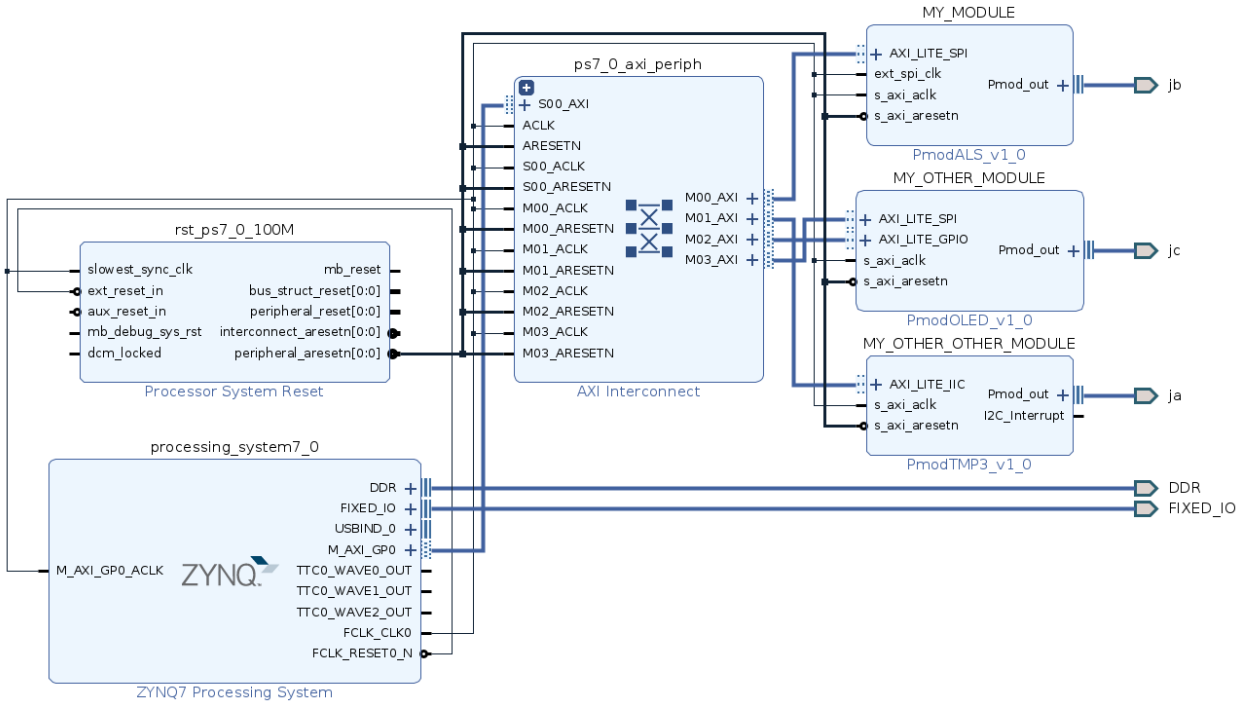
Cores | Interfaces

Search:

Name	AXI4	Status	License	VLNV
User Repository (/home/thieu/trash/vivado-library)				
Vivado Repository				
Alliance Partners				
Audio Connectivity & Processing				
Automotive & Industrial				
AXI Infrastructure				
AXI-Stream FIFO	AXI4, AXI4-Stream	Production	Included	xilinx.com:ip:axi_fifo_mm_s:4.2
AXI4-Stream Accelerator Adapter	AXI4, AXI4-Stream	Production	Included	xilinx.com:ip:axis_accelerator_adapter:2.1
AXI4-Stream Broadcaster	AXI4-Stream	Production	Included	xilinx.com:ip:axis_broadcaster:1.1
AXI4-Stream Clock Converter	AXI4-Stream	Production	Included	xilinx.com:ip:axis_clock_converter:1.1
AXI4-Stream Combiner	AXI4-Stream	Production	Included	xilinx.com:ip:axis_combiner:1.1
AXI4-Stream Data FIFO	AXI4-Stream	Production	Included	xilinx.com:ip:axis_data_fifo:2.0
AXI4-Stream Data Width Converter	AXI4-Stream	Production	Included	xilinx.com:ip:axis_dwidth_converter:1.1
AXI4-Stream Interconnect		Production	Included	xilinx.com:ip:axis_interconnect:2.1
AXI4-Stream Interconnect RTL	AXI4-Stream	Production	Included	xilinx.com:ip:axis_interconnect:1.1
AXI4-Stream Protocol Checker	AXI4, AXI4-Stream	Production	Included	xilinx.com:ip:axis_protocol_checker:2.0
AXI4-Stream Register Slice	AXI4-Stream	Production	Included	xilinx.com:ip:axis_register_slice:1.1
AXI4-Stream Subset Converter	AXI4-Stream	Production	Included	xilinx.com:ip:axis_subset_converter:1.1
AXI4-Stream Switch	AXI4, AXI4-Stream	Production	Included	xilinx.com:ip:axis_switch:1.1

IP-zentrierte Entwurfsmethode

Moderne Hardware-Entwürfe bestehen oft aus einer grafikbasierten Methode, wo das komplette Hardwaredesign aus sowohl eigenen IPs als auch aus fremden IPs gebaut wird. Die IPs müssen vom Benutzer verbunden werden.



Literatur

- Jürgen Reichardt, Bernd Schwarz, "VHDL-Simulation und -Synthese", Walter de Gruyter GmbH, 2020
- Janick Bergeron, "Writing Testbenches: Functional Verification of HDL Models", Springer Science & Business Media, 2012.
- Peter J. Ashenden, "The Designer's Guide to VHDL", Morgan Kaufmann, 2010.